Hands-On Lab: Protect Routes & Add Roles

Backend Internship — ASP.NET Core Web API — Authentication & Authorization Testing (Postman)

---

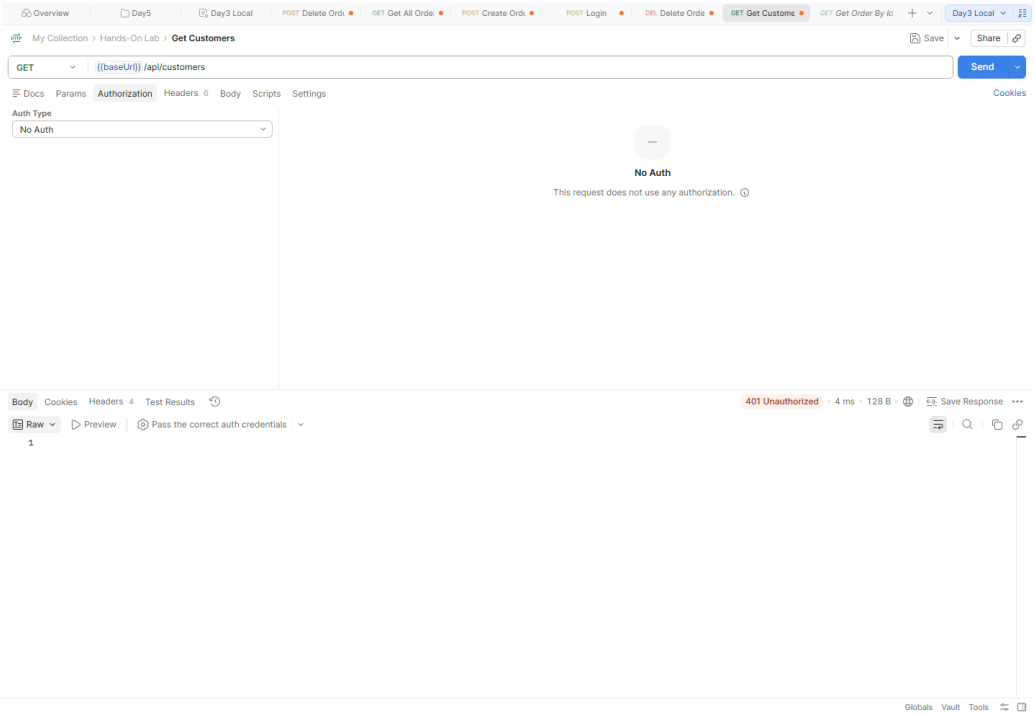
1. Protect the CRUD Controller with [Authorize]

Implementation

The Week 3 CRUD controller was protected by adding the **[Authorize]** attribute, requiring a valid JWT for all requests.

Test Result

A GET request to **/api/customers** was sent without any authentication (Auth Type: No Auth). The API returned **401 Unauthorized**, confirming the endpoint is protected.



GET /api/customers without a token → 401 Unauthorized

2. Create User and Admin Roles

Implementation

The **User** and **Admin** roles were created using ASP.NET Core Identity's **RoleManager**, and each role was assigned to a separate test user via **UserManager** (*AddToRoleAsync*). No dedicated screenshot is available for this step, as role creation was performed through seeding/Identity setup rather than a Postman request.

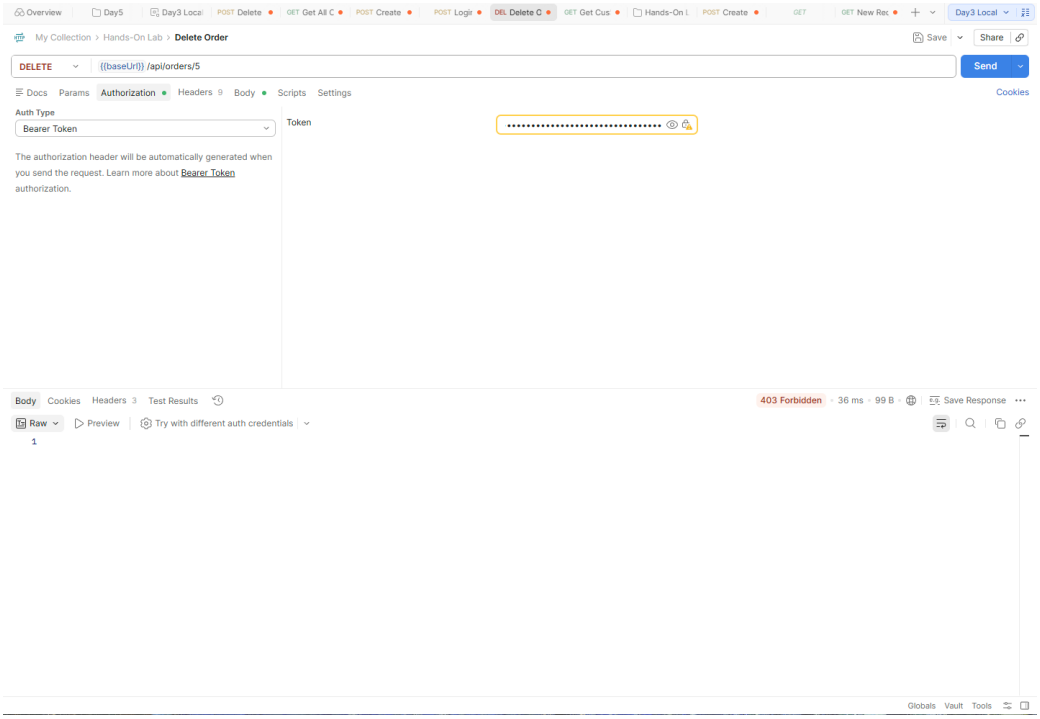
3. Restrict Delete Endpoint to Admin Role

Implementation

The Delete endpoint was restricted using **[Authorize(Roles = "Admin")]**, so only users assigned the Admin role can access it.

Test Result

A DELETE request to **/api/orders/5** was sent using a valid Bearer Token belonging to a **User**-role account. The request was authenticated but returned **403 Forbidden**, confirming the endpoint correctly rejects users without the Admin role.



DELETE /api/orders/5 with a User-role token → 403 Forbidden

4. Define and Apply a Named Authorization Policy

Implementation

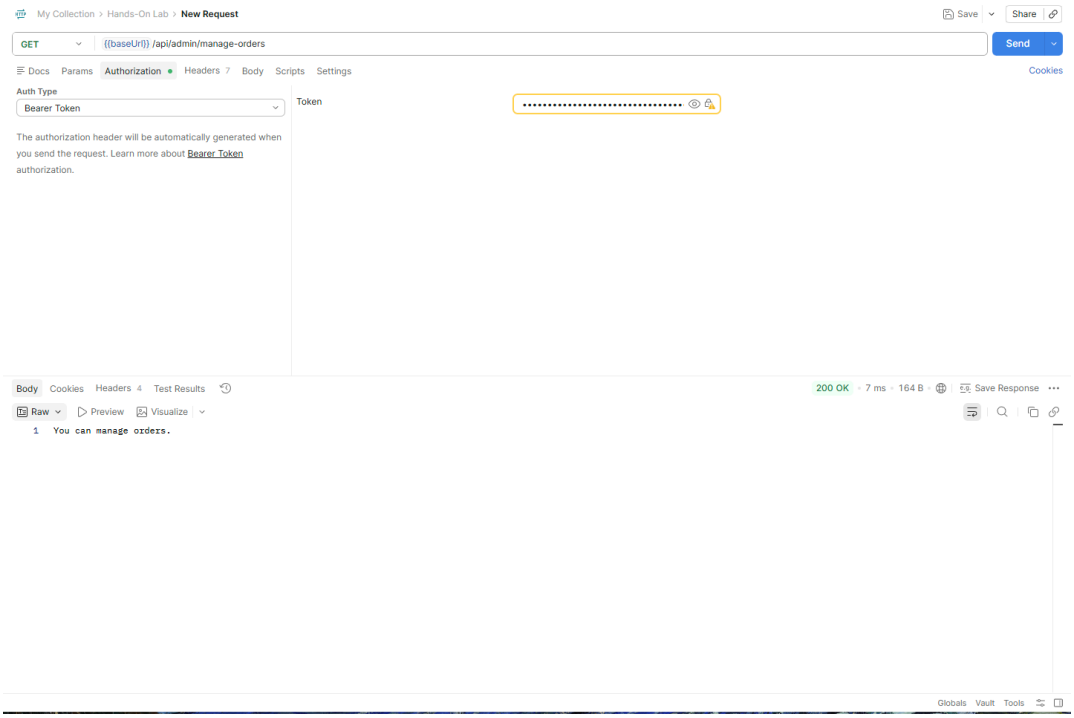
A named policy, **"CanManageOrders"**, was defined based on a Permission claim:

```
Permission = ManageOrders
```

The policy was applied to the **/api/admin/manage-orders** endpoint using **[Authorize(Policy = "CanManageOrders")]**.

Test Result

A GET request to **/api/admin/manage-orders** was sent with a Bearer Token belonging to a user holding the required permission claim. The API returned **200 OK** with the body **"You can manage orders."**, confirming the policy was correctly evaluated and enforced.



GET /api/admin/manage-orders → 200 OK, "You can manage orders."

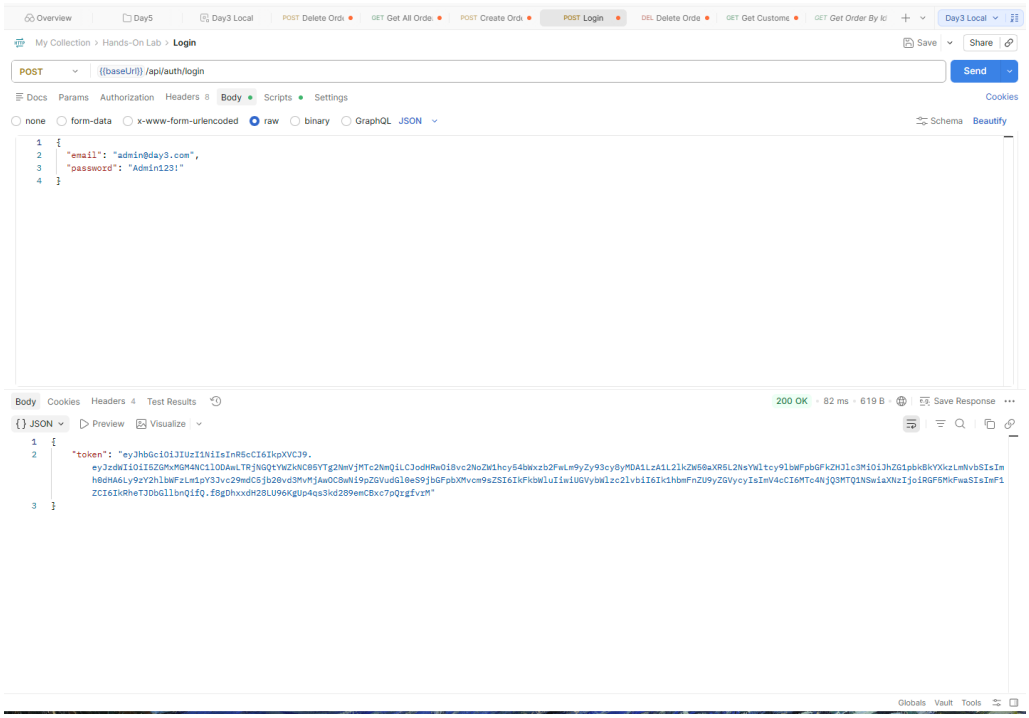
5. Postman Environment: Capture and Reuse the Login Token

Implementation

A Postman environment was set up with the variables **{{baseUrl}}** and **{{token}}**. The JWT returned by the Login request is stored in the **{{token}}** environment variable and reused as a **Bearer Token** for subsequent protected requests, avoiding manual copy-paste of the token between requests.

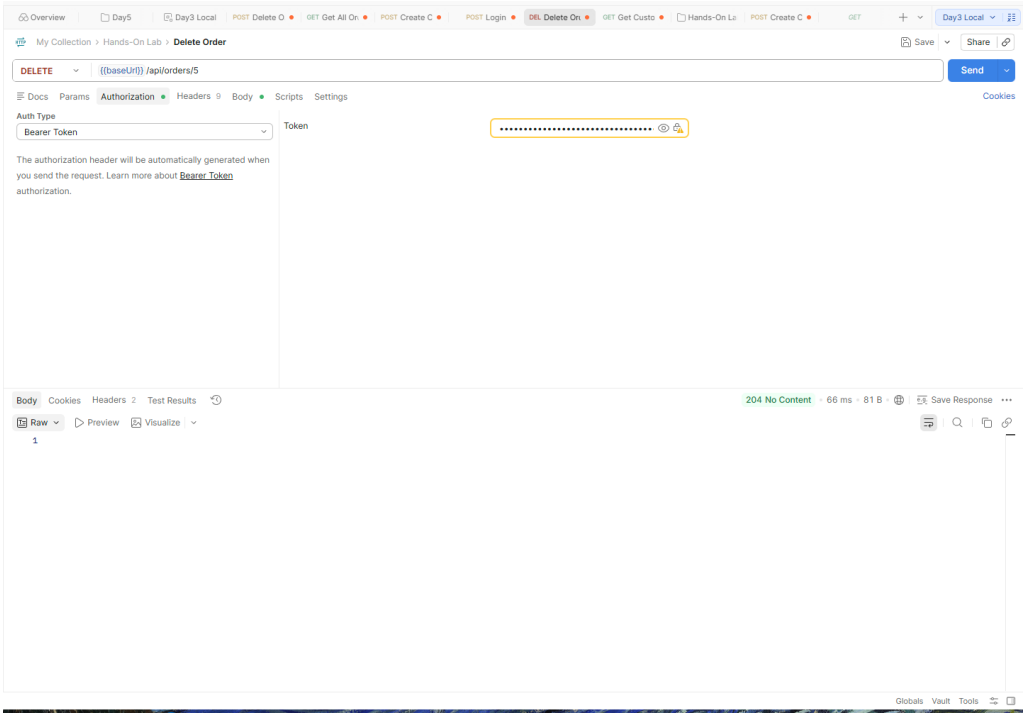
Test Result

A POST request to **{{baseUrl}}/api/auth/login** returned **200 OK** with a JWT in the response body, which was captured into the **{{token}}** variable.



POST {{baseUrl}}/api/auth/login → 200 OK with JWT token

The stored token was then reused as a Bearer Token on a protected request (DELETE **{{baseUrl}}/api/orders/5**), which returned **204 No Content**, confirming the token was applied automatically and accepted by the API.



DELETE {{baseUrl}}/api/orders/5 using the stored {{token}} → 204 No Content

Conclusion

Protected routes, User/Admin roles, Admin-only authorization, policy-based authorization ("CanManageOrders"), and Postman token reuse via environment variables were implemented and tested successfully. All Postman responses shown above confirm the expected behavior: 401 Unauthorized without a token, 403 Forbidden for insufficient role, and 200 OK / 204 No Content for authorized requests.